

CS 415 Operating Systems Spring 2026

Project #1 Report Collection

Author:

Lucas Lee Bixby

UO ID: lbixby

952001359

Report

Introduction

In this project, I was tasked with creating a basic shell environment for UNIX commands. This program allows the user to execute the program in one of two different execution styles; Interactive mode and File mode. Interactive mode allows the user to navigate and manage their file systems interactively through the command line interface. Whereas File mode reads each line of an input file, tokenizes each command by the “;” delimiter, then parses the command into arguments based on its command type. The required eight UNIX commands that were implemented as a part of this project were: ls, pwd, mkdir, cd, cp, mv, rm, and cat.

There were two provided files at the beginning of the project `string_parser.h` and `command.h`. These base header files allowed me to identify the general structure, direction, and context I needed to move forward in completing the project cleanly and efficiently. I ended up creating three new code files in the directory of `project_1`, these files were: `string_parser.c`, used to provide functionality to the `tokenize` and `free` functions defined in the `string_parser.h` header file. `Command.c`, which is used to implement each UNIX command defined in the header file of `command.h`. And finally `main.c`, which effectively determined the mode in which the program was executed, along with handling the two modes; Interactive and File.

Background

There are three vital functions at the heart of this program; those of which being, `string_tokenize()`, `parse_args()`, and `handle_command()`. The combination of these functions is how I chose to implement both interactive mode and file mode. Each mode uses a pipeline involving and structured around these three functions.

As for a brief and general explanation of the strategy I adopted, the `string_tokenize()` function is used to divide the input line (whether from file mode or interactive) into an array of individual commands separated by the “;” delimiter. The result is then stored in a member variable (`command_list`) of a new `command_line` struct. Then, in both execution modes, for each element in the `command_list` array, the command string is divided into a new array of individual arguments by the `parse_args()` function, separating each individual argument by whitespace in between. Finally, the argument array (along with `arg_count`) is sent to the `handle_command()` function. This function checks the first element in the argument array (`args[0]`) and compares it to the list of command names. If it finds a match, then that command is called and executed. If a specific command expects (n) arguments, then the following (n) elements in the `args` array after the command ID are sent to that command's function call. After all arguments have been handled for that command element, the `args` array is then freed, and the algorithm continues with the next element in `command_list`.

Implementation

There were several interesting programming tricks that I used in the development of this project. I found that the foundation of this project's code is built on many file I/O operations where data is written to a buffer, then read to a destination, whether that data was displayed or not.

The first simple example of a technique that I discovered while completing this project was the use of the `dup2()` function. My motivation in using this was to find a way to capture the standard output generated by the command functions, and write them to the specified `output.txt` file, such that I didn't need to create any separate behaviour for each command function based on whether the program was running in interactive or file mode. The `dup2()` function is used to redirect file descriptors. In my case, I used it to redirect the standard output

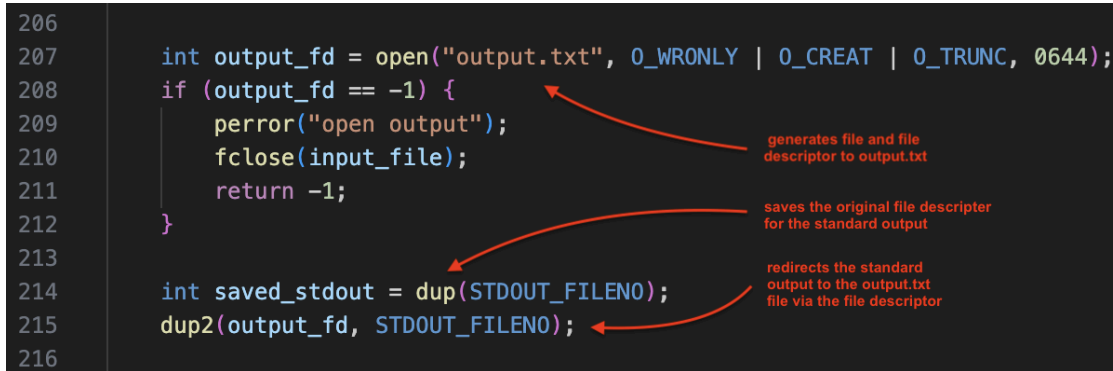
stream to the output file. The `dup2()` function takes two arguments: the source file descriptor and the destination file descriptor. It essentially causes the destination file descriptor to refer to the same open file as the source descriptor. As a result, any output written to `stdout` would be written to `output.txt` instead.

Here is a snapshot of my code where this technique was used:

```

206
207     int output_fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
208     if (output_fd == -1) {
209         perror("open output");
210         fclose(input_file);
211         return -1;
212     }
213
214     int saved_stdout = dup(STDOUT_FILENO);
215     dup2(output_fd, STDOUT_FILENO);
216

```



The diagram shows three red arrows pointing to specific lines of code with explanatory text:

- An arrow points from the text "generates file and file descriptor to output.txt" to line 207: `int output_fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);`
- An arrow points from the text "saves the original file descriptor for the standard output" to line 214: `int saved_stdout = dup(STDOUT_FILENO);`
- An arrow points from the text "redirects the standard output to the output.txt file via the file descriptor" to line 215: `dup2(output_fd, STDOUT_FILENO);`

The next interesting strategy I would like to highlight in my code is my logic inside `parse_args()`. This function essentially constructs my own `argv` array from the command string. I went about this by tokenizing each argument by whitespace delimiters (" ", "\t", "\n"). The algorithm dynamically allocates an array of string pointers, resizing it as needed using `realloc()`. The function then returns the array of argument pointers.

For this implementation, there were a few clever tricks I used to achieve the desired outcome. I set the initial capacity of the array to be 10. Then, if the count of arguments exceeded the capacity as they were being parsed, the amount of space would dynamically be updated to twice the current space. This allows for the number of potential `realloc()` operations to grow logarithmically as the number of arguments increases. This is desired behavior because reallocating memory is expensive.

Here is a snapshot from my code where this is being implemented:

```

90
91 char **parse_args(char* command_string, int *count)
92 // parse the whitespace from the command line to get elements
93 {
94     int capacity = 10;
95     char **args = malloc(capacity * sizeof(char *));
96     if (args == NULL) {
97         *count = 0;
98         return NULL;
99     }
100
101     *count = 0;
102
103     char *token = strtok(command_string, " \t\n");
104     while (token != NULL) {
105         if (*count >= capacity) {
106             capacity *= 2;
107             char **temp = realloc(args, capacity * sizeof(char *));
108             if (temp == NULL) {
109                 free(args);
110                 *count = 0;
111                 return NULL;
112             }
113             args = temp;
114         }
115
116         args[(*count)++] = token;
117         token = strtok(NULL, " \t\n");
118     }
119     return args;
120 }
121
122

```



The diagram shows four red arrows pointing to specific lines of code with explanatory text:

- An arrow points from the text "set the initial array capacity to 10" to line 94: `int capacity = 10;`
- An arrow points from the text "initialize arg count to zero" to line 101: `*count = 0;`
- An arrow points from the text "start tokenizing command string by whitespace" to line 103: `char *token = strtok(command_string, " \t\n");`
- An arrow points from the text "handle resize if count exceeds capacity" to the `if (*count >= capacity)` block starting at line 105.
- An arrow points from the text "iterate to next token" to line 117: `token = strtok(NULL, " \t\n");`

Performance Results and Discussion

The performance of my project is up to par in terms of runtime, memory allocation, and output for both interactive mode and file mode. I developed my assignment for project 1 through an SSH connection to the University of Oregon's ix-dev system, and there were no noticeable performance issues when running tests or the final program. Although I didn't have the time to optimize my code to peak its performance, my provided architecture matches the assignment details and specifications. As for memory allocation and clean up, I ran my program in interactive mode and file mode with Valgrind to check for memory leaks, and in both instances, they passed with a total of 75 allocs and 75 frees.

Here is a screenshot of my program running in file mode on the ix-dev system:

```

● lbixby@ix-dev:~/lucas_bixby_homework/lucas_bixby_assignments/CS415/project_1$ make
gcc -Wall -g -c main.c -o main.o
gcc -Wall -g -c string_parser.c -o string_parser.o
gcc -Wall -g -c command.c -o command.o
gcc -Wall -g main.o string_parser.o command.o -o pseudo-shell
● lbixby@ix-dev:~/lucas_bixby_homework/lucas_bixby_assignments/CS415/project_1$ valgrind ./pseudo-shell -f input.txt
==3668695== Memcheck, a memory error detector
==3668695== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==3668695== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==3668695== Command: ./pseudo-shell -f input.txt
==3668695==
==3668695==
==3668695== HEAP SUMMARY:
==3668695==   in use at exit: 0 bytes in 0 blocks
==3668695==   total heap usage: 75 allocs, 75 frees, 691,217 bytes allocated
==3668695==
==3668695== All heap blocks were freed -- no leaks are possible
==3668695==
==3668695== For lists of detected and suppressed errors, rerun with: -s
==3668695== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
○ lbixby@ix-dev:~/lucas_bixby_homework/lucas_bixby_assignments/CS415/project_1$

```

Finally, for the output of my program. When running interactive mode on ix-dev and following along with the demo provided in the project description, my output covers the error cases and matches the output as intended.

```

● lbixby@ix-dev:~/lucas_bixby_homework/lucas_bixby_assignments/CS415/project_1$ valgrind ./pseudo-shell
==3687788== Memcheck, a memory error detector
==3687788== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==3687788== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==3687788== Command: ./pseudo-shell
==3687788==
>>> ls
.. command.c test input.txt test_direct_1 output.txt main.c .gitignore test_direct_2 string_parser.c command.o example-output-1.txt README.txt command.h Makefile
string_parser.h main.o Report_Collection_Template-1.docx pseudo-shell string_parser.o Project_1 - Description.pdf
>>> ls ;
.. command.c test input.txt test_direct_1 output.txt main.c .gitignore test_direct_2 string_parser.c command.o example-output-1.txt README.txt command.h Makefile
string_parser.h main.o Report_Collection_Template-1.docx pseudo-shell string_parser.o Project_1 - Description.pdf
>>> ls ; ls
.. command.c test input.txt test_direct_1 output.txt main.c .gitignore test_direct_2 string_parser.c command.o example-output-1.txt README.txt command.h Makefile
string_parser.h main.o Report_Collection_Template-1.docx pseudo-shell string_parser.o Project_1 - Description.pdf
.. command.c test input.txt test_direct_1 output.txt main.c .gitignore test_direct_2 string_parser.c command.o example-output-1.txt README.txt command.h Makefile
string_parser.h main.o Report_Collection_Template-1.docx pseudo-shell string_parser.o Project_1 - Description.pdf
>>> ls ; ls ; test
.. command.c test input.txt test_direct_1 output.txt main.c .gitignore test_direct_2 string_parser.c command.o example-output-1.txt README.txt command.h Makefile
string_parser.h main.o Report_Collection_Template-1.docx pseudo-shell string_parser.o Project_1 - Description.pdf
.. command.c test input.txt test_direct_1 output.txt main.c .gitignore test_direct_2 string_parser.c command.o example-output-1.txt README.txt command.h Makefile
string_parser.h main.o Report_Collection_Template-1.docx pseudo-shell string_parser.o Project_1 - Description.pdf
Error! Unrecognized command: test
>>> ls ; test ; ls
.. command.c test input.txt test_direct_1 output.txt main.c .gitignore test_direct_2 string_parser.c command.o example-output-1.txt README.txt command.h Makefile
string_parser.h main.o Report_Collection_Template-1.docx pseudo-shell string_parser.o Project_1 - Description.pdf
Error! Unrecognized command: test
.. command.c test input.txt test_direct_1 output.txt main.c .gitignore test_direct_2 string_parser.c command.o example-output-1.txt README.txt command.h Makefile
string_parser.h main.o Report_Collection_Template-1.docx pseudo-shell string_parser.o Project_1 - Description.pdf
>>> ls ; ls
Error! Unsupported parameters for command: ls
>>> ls ; ls test
.. command.c test input.txt test_direct_1 output.txt main.c .gitignore test_direct_2 string_parser.c command.o example-output-1.txt README.txt command.h Makefile
string_parser.h main.o Report_Collection_Template-1.docx pseudo-shell string_parser.o Project_1 - Description.pdf
Error! Unsupported parameters for command: ls
>>> ls ; ls
.. command.c test input.txt test_direct_1 output.txt main.c .gitignore test_direct_2 string_parser.c command.o example-output-1.txt README.txt command.h Makefile
string_parser.h main.o Report_Collection_Template-1.docx pseudo-shell string_parser.o Project_1 - Description.pdf
.. command.c test input.txt test_direct_1 output.txt main.c .gitignore test_direct_2 string_parser.c command.o example-output-1.txt README.txt command.h Makefile
string_parser.h main.o Report_Collection_Template-1.docx pseudo-shell string_parser.o Project_1 - Description.pdf
>>> exit
==3687788==
==3687788== HEAP SUMMARY:
==3687788==   in use at exit: 0 bytes in 0 blocks
==3687788==   total heap usage: 76 allocs, 76 frees, 1,450,142 bytes allocated
==3687788==
==3687788== All heap blocks were freed -- no leaks are possible
==3687788==
==3687788== For lists of detected and suppressed errors, rerun with: -s
==3687788== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
○ lbixby@ix-dev:~/lucas_bixby_homework/lucas_bixby_assignments/CS415/project_1$

```

Conclusion

Overall, this project was really interesting in how we utilized system calls to interact directly with the computer's OS. I definitely learned a lot of useful skills and techniques that I will be able to use in similar projects moving forward. I think that if I were to come back and revisit this project in the future. I would add the following functionalities: auto-fill path/filenames with tab, more file management commands like "touch" to create files and "rm -r" to delete repositories along with their contents, and finally, possibly a way to write and directly edit a file in interactive mode like you would in Vim.